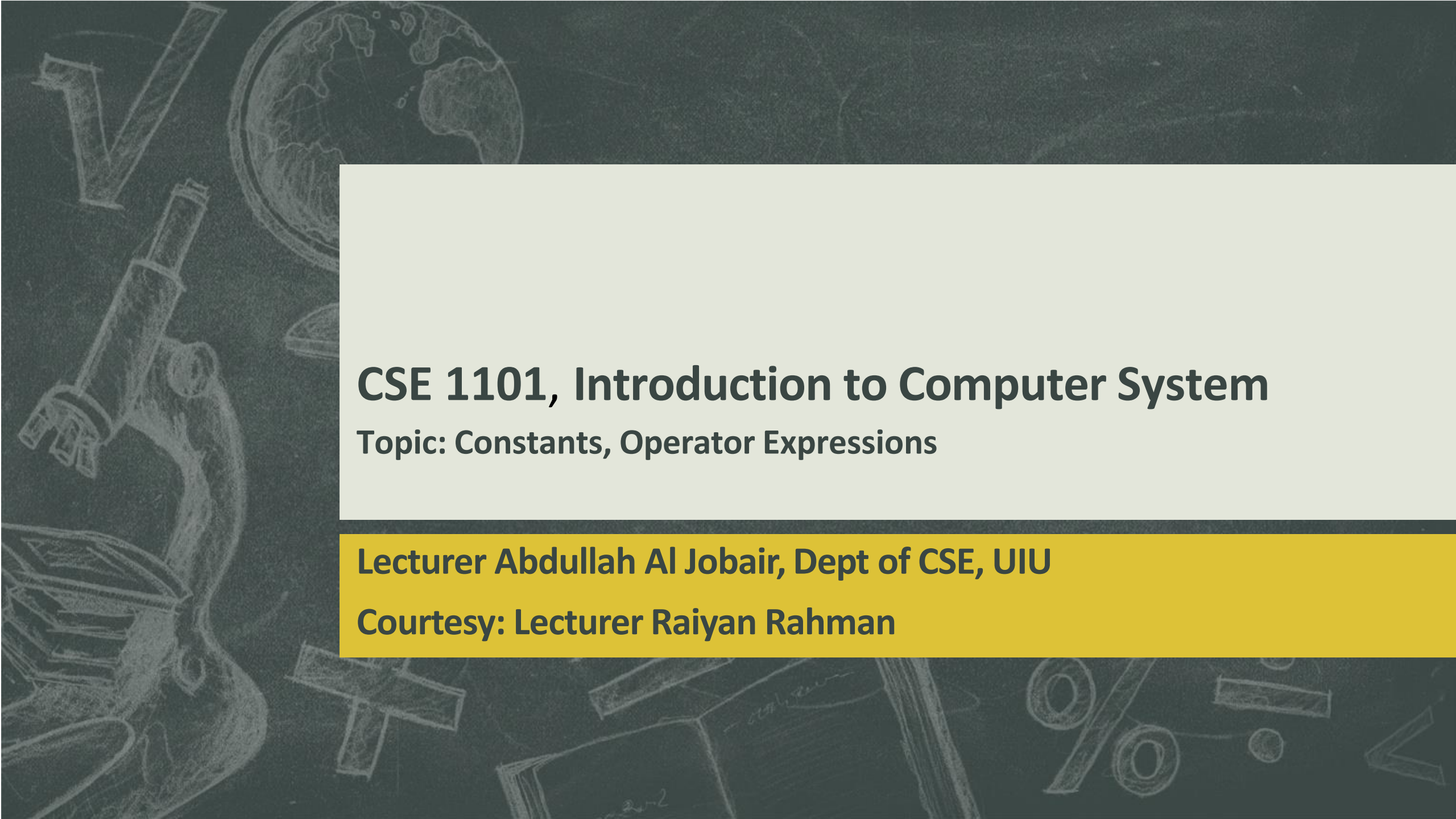




CSE 1101, Introduction to Computer System

Topic: Constants, Operator Expressions

Lecturer Abdullah Al Jobair, Dept of CSE, UIU

Courtesy: Lecturer Raiyan Rahman

Variable vs Constant

- **Variable:** A variable is nothing but a name given to a storage area.
 - A variable may take different values at different times during execution.

Variable declaration: **data_type** variable_name;

data_type variable_name=value;

- **Constant:** fixed value that do not change during the execution of a program.

Constant declaration: **#define** constant_name **value**

Rules for Defining Constant

- Naming rule is the same as naming a variable.
- `#define` must be written before defining a constant
- **At least one blank** space between `#define` and **symbolic name**
and between the **symbolic name and constant**.
- `#define` statement **must not end with a semicolon**
- After defining the symbolic name should not be assigned any other value in the program
- `#define` statement may appear anywhere in the program before use.

Valid or Invalid

- #define A=21 invalid
- #define A 20; invalid
- #Define A 15 invalid
- #define A 11 valid

Example Code to Define a Constant

```
#include<stdio.h>
#define PI 3.142

int main(){
    float radius, area;
    printf("Take input of radius:");
    scanf("%f",&radius);

    area =PI*radius*radius;
    printf("Area of circle=%f\n",area);

    printf("Area of circle=%0.3f\n",area);
}
```

▪ Output?

Take input of radius:

3

Area of circle=28.278000

Area of circle=28.278

Operators

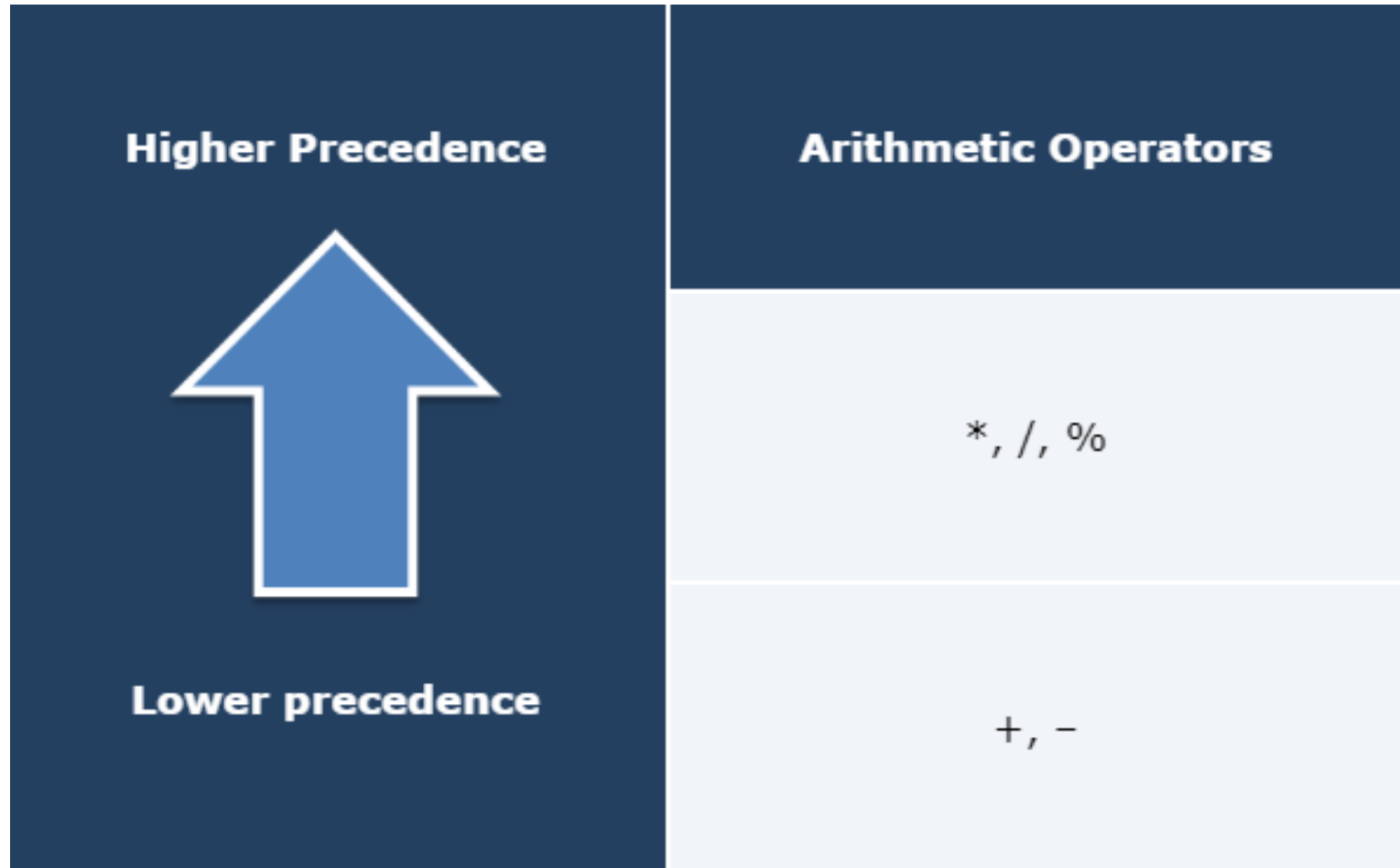
- **Operator:** Operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.
- C operators can be classified into a number of categories:
 - Arithmetic operators
 - Relational operators
 - Logical operators
 - Assignment operators
 - Increment and decrement operators
 - Bitwise operators
 - Special Operators

Arithmetic Operators

Arithmetic Operator	C Operation	Example	Possible operands
+	Addition	5+6, a+b, a+4	Any number
-	Subtraction	6.5-5.0, a-b	Any number
*	Multiplication	6*5.5, a*b	Any number
/	Division	6/2, a/b	Any number
%	Modulus	14%3	Only Integer number

Operator Precedence

- When **multiplication** and **division** exist in the same operation, as **both are of the same precedence**, they are **performed left to right**.
- Which means, the expression $6/3*2$ is equivalent to the expression $(6/3)*2$



Operator Precedence

- Any operations enclosed in parentheses are performed first.
- Any multiplication, division and modulus performed next from left to right
- Then addition, subtraction performed from left to right

Example 2 - Age Calculator

Here's what the output should look like:

 "E:\Lec MIST\CLASS\Term II 2020\CSE 110\age_calculator.exe"

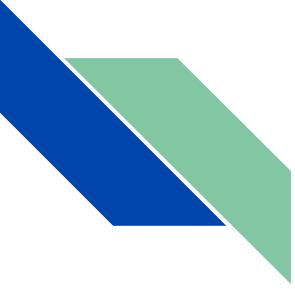
Enter the year you were born: 1999

Enter the year to calculate your age in: 2055

You'll be 56 years old in 2055

Process returned 0 (0x0) execution time : 10.192 s

Press any key to continue.



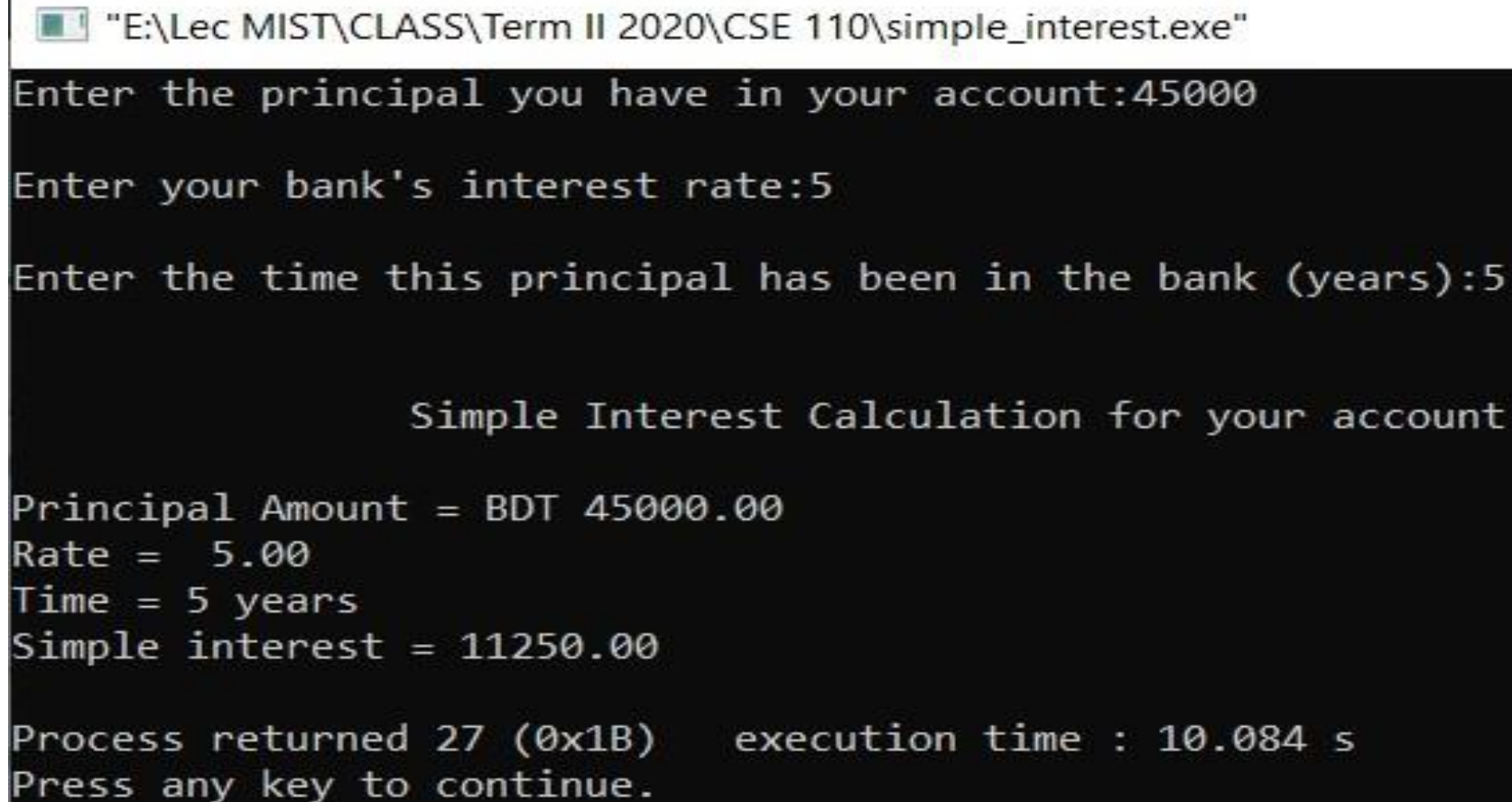
Example 2 - Age Calculator

Let's try out the steps for writing programs!

Step 1	Step 2
Ask the user when he/she was born	printf()
Store their answer (in birth_year)	scanf() + use an int variable
Ask and store the year he/she wants to calculate the age in (in calc_year)	printf() + scanf() + use an int variable
Do calc_year - birth_year to find age	Arithmetic operation (-)
Show them the result!	printf()

Example 3 - Simple Interest

Make a simple interest calculator for bank accounts that takes the principal, interest rate and time as input and finds the interest accordingly. [Check out the solution in simple_interest.c](#)

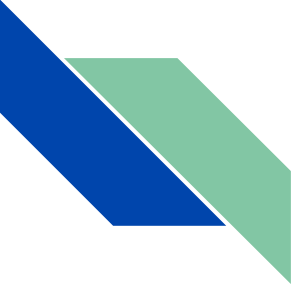


```
"E:\Lec MIST\CLASS\Term II 2020\CSE 110\simple_interest.exe"
Enter the principal you have in your account:45000
Enter your bank's interest rate:5
Enter the time this principal has been in the bank (years):5

          Simple Interest Calculation for your account

Principal Amount = BDT 45000.00
Rate = 5.00
Time = 5 years
Simple interest = 11250.00

Process returned 27 (0x1B)    execution time : 10.084 s
Press any key to continue.
```



Example 3 - Simple Interest

Let's try out the steps for writing programs!

Step 1	Step 2
Ask and store the principal amount	printf() + scanf() + use an int variable
Ask and store the interest rate	printf() + scanf() + use another int variable
Ask and store the time	printf() + scanf() + use an int variable
Do $(\text{principal} * \text{rate} * \text{time}) / 100.0$ to find simple interest	Arithmetic operation + store in an int variable
Show the result!	printf()

Example 4 - Worker's Salary

Let's calculate a garment (or any) worker's salary calculator. Takes in daily rate and days worked, then calculates monthly salary. Then also takes in overtime hourly rate and hours worked to find salary with overtime. [Check out the solution in intro_salary.c](#)

```
"E:\Lec MIST\CLASS\Term II 2020\CSE 110\Codes\intro_salary.exe"

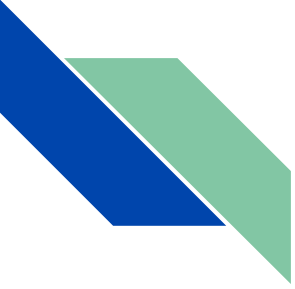
Salary Calculator
Developed in CSE 106 Sessional
MIST 2020

Enter your basic salary per day: 450
Enter how many days you worked last month: 26

Your basic salary for last month is: 11700.00
How many hours did you work overtime last month? : 24

Enter your overtime rate: 85

Your basic salary for last month (with overtime pay) is: 13740.00
Process returned 0 (0x0)    execution time : 30.558 s
Press any key to continue.
```



Example 4 - Worker's Salary

Let's try out the steps for writing programs!

Step 1	Step 2
Ask and store the basic daily salary	printf() + scanf() + use a float variable
Ask and store how many days the worker worded last month	printf() + scanf() + use an int variable
Calculate the Basic salary and show	Arithmetic operation + printf()
Ask and store overtime hours worked by the worker last month	printf() + scanf() + use an float variable
Ask and store overtime rate per hour	printf() + scanf() + store in a float variable
Calculate the OT salary (basic + overtime salary) and show.	Arithmetic operation + printf()

Relational Operators

Relational Operator	C Operation	Example
>	Greater Than	<code>x > y</code>
<	Less Than	<code>x < y</code>
>=	Greater Than or Equal	<code>x >= y</code>
<=	Less Than or Equal	<code>x <= y</code>
==	Equal	<code>x == y</code>
!=	Not Equal	<code>x != y</code>

Logical Operators

Logical Operator	C Operation	Structure	Example
&&	Logical AND	Condition1 && Condition2 (both condition must be true)	<code>x > 1 && y > 5</code>
	Logical OR	Condition1 Condition2 (atleast one condition must be true)	<code>x > 1 y > 5</code>
!	NOT		<code>!(x > 1)</code>

- Relational and Logical operators **return value 1** if the expression is **true**
- And **return value 0** if the expression is **false**

Relational & Logical Operators

Expression	True/False	Return Value
9>7	true	1
9<7	false	0
9>7 && 9!=7	true	1
9>7 9!=7	true	1
9<7 && 9!=7	false	0
9<7 9!=7	true	1
!(9==7)	true	1

Assignment Operator

Operator Name	sign	example	Meaning in C
Assignment	=	A = 5+6; A =x+y; A = 5; A = x;	Assigns the value of the right expression or Variable to the left operand

Increment & Decrement Operators

- Let X=5;

Operator Name	Type	example	Use in C program	Meaning in C
Increment	Prefix	++X	A = ++X	1.Increments the value of X by 1 2.Assigns the value to A
	Postfix	X++	A = X++	1.Assigns the current value of X to A 2.Increment the value of X by 1
Decrement	Prefix	--X	A = --X	1.Decrements the value of X by 1 2.Assigns the value to A
	Postfix	X--	A = X--	1.Assigns the current value of X to A 2.Decrement the value of X by 1

Bitwise Operators

- It is used for manipulating data at bit level
- Used for testing bits or shifting them right or left.

Operator	Name	Example
~	Not	~x
&	AND	x & y
	OR	x y
^	XOR	x ^ y
<<	Left shift	x << 1
>>	Right shift	x >> 1

Bitwise Operators Example Code

```
#include<stdio.h>
int main() {
    int a = 11;    /// 0000 1011
    int b=9;       /// 0000 1001
    printf("%d\n", a&b); /// by bitwise ANDing a&b 0000 1001 = 9
    printf("%d\n", a|b); /// by bitwise ORing a|b 0000 1011 = 11
    printf("%d\n", a^b); /// by bitwise XORing a^b 0000 0010 = 2
    printf("%d\n", a<<1); /// by left-shifting a to 1 position 0001 0110 = 22
    printf("%d\n", a>>1); ///by right-shifting a to 1 position 0000 0101 = 5
    return 0;
}
```

Special Operator

- `sizeof` operator:
 - It returns the number of bytes the operand occupies.
 - `i=sizeof(int);`

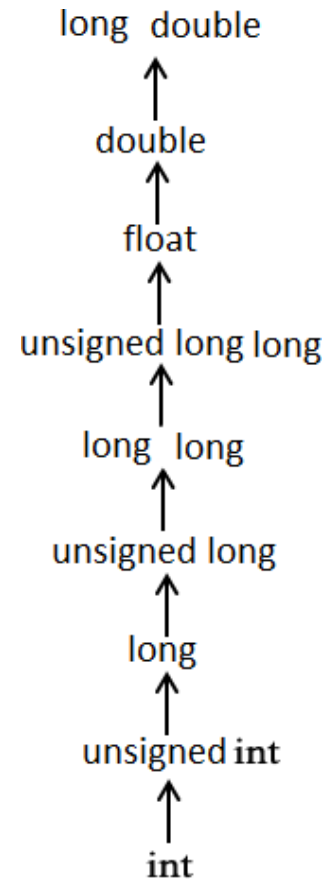
Special Operator

```
#include<stdio.h>
int main() {
    int a = sizeof(int);
    printf("Number of bytes occupied by an integer operand a is:%d\n", a);
}
```

Type Cast

- **Converting** one datatype into another is known as type casting or, type-conversion. Two types of conversion:
 - **Implicit conversion** : If the operands are of different type the lower type is automatically converted to the higher type before the operation proceeds. The result is of higher type. **Example:** A float type variable will be converted to double type automatically
 - **Explicit conversion** : Conversion from higher type to lower type requires explicit conversion. **Example:** Double to float conversion/ Float to int conversion
- You can convert the values from one type to another explicitly using the **cast operator** : **(type_name) expression**

Implicit Type Conversion Example



Type Cast

Example	Action
<code>X = (int) 3.5</code>	3.5 is converted to integer
<code>X = (int)(a+b)</code>	The result of (a+b) is converted to integer
<code>X = (int)a+b</code>	a is converted to integer and then added with b
<code>X = (int)a/ (int)b</code>	a is converted to int, b is converted to int, then division operation performed
<code>X = (int)20.5 / (int)4.33</code>	20.5 converted to floor int(20), 4.33 converted to floor int(4), then division operation performed

Example 5 - Typecasting

Let's try to divide one integer number by another (i.e 27/5) and try to store the result in a float variable. Would we be alright without typecasting?

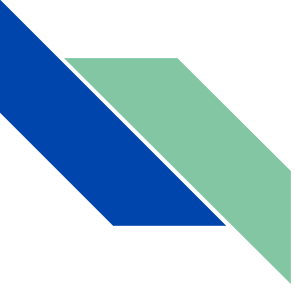
```
"E:\Lec MIST\CLASS\Term II 2020\CSE 110\Codes\typecasting_Example.e
Enter First number (dividend): 27

Enter second number (divisor): 5

Res (quotient) of 27 / 5 is: 5.40

Process returned 0 (0x0)    execution time : 5.654 s
Press any key to continue.
```

[Check out the solution in typecasting_example.c](#)



Example 5 - Typecasting

Let's try out the steps for writing programs!

Step 1	Step 2
Ask and store the num1(dividend)	printf() + scanf() + use a int variable
Ask and store the num 2 (divisor)	printf() + scanf() + use an int variable
Calculate the quotient	Arithmetic operation (a/b)
But we need to convert to float since the result of division of two integers may be decimal!	Have to Use typecasting!
Show the result of div (quotient)	printf()

Example 6 - A Gold Shop!

Let's make a program that calculates the price of a gold ornament to be purchased by the customer. It takes input of purchased amount of gold, today's rate/bhori and discount rate. Then, Vat (15%) and the entered discount to be given to the customer. Final price should also be "floored" to the nearest hundredth value. Also, one tk per purchase to be donated.

```
"E:\Lec MIST\CLASS\Term II 2020\CSE 110\Codes\gold_shop.exe"

!!MIST Gold Shop!!
Checkout Menu

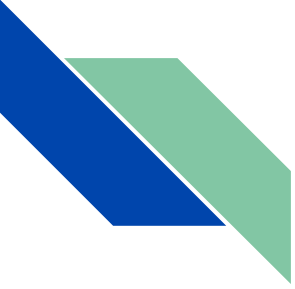
today's price per bhorì: 54400
Today's special discount rate: 5 %
Enter the weight of Gold bought by customer (in bhorì): 2.5544

Price: 151814.093750

****5% discount given and 15% vat added to the price. And 1 tk donated!****

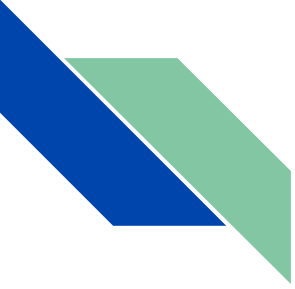
Price for the customer (discard after decimal (no paisas!), floored to nearest 100!): 151800

Process returned 0 (0x0)   execution time : 5.078 s
Press any key to continue.
```



Example 6 - A Gold Shop!

Step 1	Step 2
Take input of the daily gold rate (per bhor) and today's discount rate	printf() + scanf() + use float variables
Take input from shopkeeper how much gold bought (in bhor)	printf() + scanf() + use a float variable
Calculate the raw price	Arithmetic operation (gold_rate*weight_bought)
Give discount according to rate	Arithmetic operation + typecasting
Add 15% VAT, donate 1 tk	Arithmetic operation
Show the actual price	printf()
Calculate and show the price for the customer (discard all paisas, floor to nearest 100!)	Explicit typecasting + Arithmetic operation + printf()



Escape Sequences for printf()

Escape Sequence	What it does
<code>\n</code>	Prints a new line. More specifically, moves the cursor to a new line.
<code>\t</code>	Moves the cursor one tab to the right.
<code>\r</code>	Moves the cursor to the beginning of the current line. Then whatever you write after <code>\r</code> will overwrite the previous characters. This is called carriage return.
<code>\b</code>	moves the cursor back one space. Then whatever you write overwrites what you had in that space. However, it can't go back to a previous line.
<code>\a</code>	Doesn't move the cursor. Just makes a “beep” or windows “alert” sound for that line.
<code>\</code>	Put <code>\</code> before any of the above escape sequences from above if you want to print the escape sequence itself rather than to move the cursor.
<code>%%</code>	However, if you want to print <code>%</code> itself just put another <code>%</code> before that. Putting <code>\</code> before <code>%</code> won't work.

Note: Escape characters take effect within that `printf()` (that line) only.

A Simple Task For Practice – Do yourself

Calculate the area of geometric shapes (circle, rectangle, square, triangle). Write separate programs for each shape's area calculation.



"E:\Lec MIST\CLASS\Term II 2020\CSE 110\shape_calc.exe"

```
Which Shape's Area do you want to calculate?
```

```
1 --> Circle
```

```
2 --> Rectangle
```

```
3 --> Triangle
```

```
4 --> Square
```

```
Enter an option: 3
```

```
Enter the base and height
```

```
5 10
```

```
Area of a Triangle = 25.000000
```

```
Process returned 31 (0x1F)    execution time : 7.771 s
```

```
Press any key to continue.
```

Math.h header file

Math.h header file

function	Library function	remark
$\cos x$	<code>cos(x)</code>	computes the cosine of an argument. x is in radian
$\sin x$	<code>sin(x)</code>	computes the sine of an argument. x is in radian
$\sqrt{x}$	<code>sqrt(x)</code>	computes square root of a number
$\sqrt[3]{x}$	<code>cbrt(x)</code>	computes cube root of a number
e^x	<code>exp(x)</code>	computes the exponential raised to the argument
$ x $	<code>fabs(x)</code>	computes absolute value. For float
$ x $	<code>abs(x)</code>	computes absolute value. For int
x^y	<code>pow(x,y)</code>	Computes power of a number
$\log_e x / \ln x$	<code>log(x)</code>	Computes natural logarithms
$\log_{10} x$	<code>log10(x)</code>	Computes 10 base logarithms

Math.h header file

Example:

$$\sqrt[4]{x} = x^{1/4} = x^{.25} \rightarrow \text{pow}(x, .25)$$

$$\frac{1}{x^4} = x^{-4} \rightarrow \text{pow}(x, -4)$$

*Thank
you!*